Taller 3

Juan Camilo Panadero, Sergio Santana y Sergio Sánchez

Punto 1. Probabilidades y utilidad esperada

a) Tabla conjunta de sensor_basico a mano

Datos: P(I=1) = 0,10, P(A=1 | I=1) = 0,90 y P(A=1 | I=0) = 0,05, donde I es Incendio y A es Alarma. Cada fila se obtiene con la regla del producto P(I, A) = P(I) P(A | I).

Incendio	Alarma	Cálculo	P(I, A)
0	0	0,90 × 0,95	0,855
0	1	0,90 × 0,05	0,045
1	0	0,10 × 0,10	0,010
1	1	0,10 × 0,90	0,090
Suma			**1,000**

P(A=1) se obtiene sumando las filas con alarma: 0,045 + 0,090 = 0,135. Con la regla de Bayes:

$$P(I=1 \mid A=1) = P(A=1 \mid I=1)\,P(I=1) / P(A=1) = 0{,}090 / 0{,}135 = 0{,}6667$$

Recibir la alarma sube la probabilidad de incendio de 0,10 a 0,67. El programa da el mismo resultado en el punto 1 con el caso alarma (Incendio=1: 0,666667), lo que confirma la tabla.

b) y c) Implementación

normalize valida primero la entrada: rechaza el diccionario vacío, los pesos que no son números, los negativos, los infinitos o NaN y una suma igual a cero. Luego divide cada peso entre el total y retorna un diccionario nuevo con las mismas claves y en el mismo orden, sin tocar el original. La suma se hace con math.fsum para reducir el error de redondeo.

expected_utility calcula la suma de P(s) U(s) sobre los resultados. best_action recorre las acciones en el orden de la tabla, calcula la utilidad esperada de cada una y solo cambia la acción elegida cuando aparece una estrictamente mayor. Así, en un empate se conserva la primera acción, como pide el enunciado.

d) Comparación de casos

Caso	P(I=1 | e)	EU (monitorear)	EU (inspeccionar)	EU (brigada)	Acción
sin_evidencia	0,1000	−10,000	−9,700	−27,500	inspeccionar
alarma	0,6667	−66,667	−19,333	−13,333	brigada
sin_alarma	0,0116	−1,156	−8,197	−29,711	monitorear

Ejemplo del cálculo con alarma: EU(brigada) = 0,333 × (−30) + 0,667 × (−5) = −13,333. Cada evidencia lleva a una acción distinta. La discusión está en el punto 6a, literal a.

Punto 2. Inferencia exacta por enumeración

a) Implementación

enumeration_ask usa una función auxiliar recursiva que recorre network.variables en orden topológico. Si la variable ya tiene valor en el evento, multiplica su probabilidad local; si es oculta, suma los dos casos (0 y 1) extendiendo una copia del evento. Al final del recorrido el producto vale 1. Se calcula un peso para cada valor de la consulta y se normaliza con normalize. Si la consulta está observada, el valor contrario recibe peso cero y el resultado es una distribución puntual. Si los dos pesos suman cero, la evidencia es imposible y se lanza ValueError. La evidencia nunca se modifica porque siempre se trabaja sobre copias.

b) Resultados

En sensor_basico con alarma, la enumeración da P(Incendio=1 | Alarma=1) = 0,666667, igual al cálculo manual del punto 1. En bosque:

Caso (bosque)	Evidencia	P(Incendio=1 | e)
sin_evidencia	ninguna	0,0652
alarma	Alarma=1	0,3295
alarma_falla	Alarma=1, Falla=1	0,0768

Sin evidencia el incendio es poco probable (6,5 %). La alarma lo sube a casi 33 %, porque la alarma depende de humo y temperatura, que a su vez dependen del incendio. Cuando además se sabe que el sensor falló, la probabilidad baja a 7,7 %: la falla ya explica por qué sonó la alarma, así que el incendio deja de ser necesario para explicarla. Es el efecto de explicación competitiva, porque Alarma es un colisionador entre las causas del humo y la temperatura por un lado y Falla por el otro.

c) Independencia en cadena_humo

La red es la cadena Sequedad → Incendio → Humo. El único camino entre Sequedad y Humo pasa por Incendio, y en una cadena ese camino se bloquea cuando el nodo del medio se observa. Por d-separación, Humo es independiente de Sequedad dado Incendio. Además, la CPT de Humo solo tiene a Incendio como padre, así que una vez se fija Incendio la sequedad no aparece en el cálculo.

Consulta	Resultado
P(Humo=1 | Incendio=1, Sequedad=0)	0,8000
P(Humo=1 | Incendio=1, Sequedad=1)	0,8000
P(Humo=1 | Sequedad=0)	0,1140
P(Humo=1 | Sequedad=1)	0,3100

Las dos primeras filas coinciden, lo que confirma la independencia condicional. Las dos últimas muestran que sin observar Incendio el camino queda activo: la sequedad hace más probable el incendio (0,30 frente a 0,02) y el incendio hace más probable el humo. Por eso la igualdad dado Incendio no implica que Humo y Sequedad sean independientes en general.

Punto 3. Eliminación de variables e inferencia aproximada

a) P(Humo=1) en cadena_humo a mano

Factores iniciales: P(S), P(I | S) y P(H | I). Orden: primero Sequedad y luego Incendio.

Paso 1. Eliminar Sequedad. Se multiplican P(S) y P(I | S) y se suma sobre S, lo que deja un factor sobre Incendio:

Incendio	$f_1(I) = \Sigma_s P(s) P(I \mid s)$	Valor
0	0,6 × 0,98 + 0,4 × 0,70	0,868
1	0,6 × 0,02 + 0,4 × 0,30	0,132

Paso 2. Eliminar Incendio. Se multiplican $f_1(I)$ y P(H | I) y se suma sobre I, lo que deja un factor sobre Humo:

Humo	$f_2(H) = \Sigma_i f_1(i) P(H \mid i)$	Valor
0	0,868 × 0,90 + 0,132 × 0,20	0,8076
1	0,868 × 0,10 + 0,132 × 0,80	0,1924

No hay evidencia, así que P(Humo=1) = 0,1924 sin normalizar. El programa, con eliminación de variables y orden topológico, da el mismo valor y la misma traza (f_1 = 0,868 / 0,132). El mayor factor producto tiene 4 filas.

b) Órdenes de eliminación en bosque con alarma

Método	Orden de eliminación	P(Incendio=1 | Alarma=1)	Mayor factor producto
Enumeración	no aplica	0,329517	no aplica
Eliminación, forward	Tormenta, Sequedad, Falla, Humo, Temperatura	0,329517	8 filas
Eliminación, reverse	Temperatura, Humo, Falla, Sequedad, Tormenta	0,329517	32 filas

Los tres métodos coinciden. Con el orden inverso el primer paso elimina Temperatura, que aparece junto a Incendio, Sequedad, Humo y Falla, y el producto queda sobre cinco variables (2 = 32 filas). Con el orden topológico ningún producto pasa de tres variables (8 filas). La discusión está en el punto 6a, literal b.

c) y d) Implementación del muestreo

prior_sample recorre las variables en orden topológico, consulta P(X=1 | padres) con los valores ya muestreados y asigna X=1 si rng.random() es menor que esa probabilidad. Solo usa el generador recibido, así que una misma semilla reproduce las mismas muestras.

sampling_ask genera exactamente samples mundos con prior_sample. Sin rechazo cuenta el valor de la consulta en todas las muestras (la validación suministrada exige evidencia vacía). Con rechazo descarta los mundos que contradicen alguna variable de la evidencia y cuenta solo los

aceptados. Los conteos se normalizan entre las aceptadas, no entre los intentos. Si no se acepta ninguna muestra retorna (None, 0).

e) Resultados de muestreo en bosque (consulta Incendio, semilla 7)

Método y caso	Intentos	Aceptadas	Estimación P(1)	Exacto	Error absoluto
A priori, sin evidencia	1000	1000	0,0560	0,0652	0,0092
A priori, sin evidencia	10000	10000	0,0634	0,0652	0,0018
Rechazo, alarma	1000	191	0,2775	0,3295	0,0520
Rechazo, alarma	10000	1841	0,3156	0,3295	0,0139
Rechazo, rara	1000	0	sin estimación	0,2608	no aplica
Rechazo, rara	10000	11	0,0909	0,2608	0,1699

Con cinco repeticiones (semillas 7 a 11) el patrón se mantiene. A priori con 1000 intentos los errores van de 0,003 a 0,012, y con 10000 de 0,0004 a 0,003. Con rechazo y alarma se aceptan cerca del 18 % de los intentos (171 a 202 de 1000). Con el caso rara, de 1000 intentos cuatro repeticiones no aceptaron ninguna muestra y una aceptó solo una, que dio una estimación de 1,0. Con 10000 intentos se aceptaron entre 10 y 16 muestras y las estimaciones variaron entre 0,09 y 0,25. El análisis está en el punto 6a, literal c.

Punto 4. MDP e iteración de valores

a) Elementos del MDP en cuadricula

y\x	0	1	2	3
0	.	.	.	G (+1)
1	.	#	.	X (−1)
2	S	.	.	.

- **Estados:** las 11 celdas que no son muro, identificadas por su posición (x, y) con origen arriba a la izquierda. S = (0, 2) es el inicio.
- **Acciones:** N, E, S, W en las celdas no terminales; en los terminales no hay acciones.
- **Transiciones:** con ruido 0,2 el dron va en la dirección elegida con probabilidad 0,8 y en cada dirección perpendicular con 0,1. Si el movimiento choca con el muro o el borde, se queda en su celda, y las probabilidades que llevan al mismo sucesor se suman. Por ejemplo, desde S con la acción N: 0,8 a (0, 1), 0,1 a (1, 2) por el desvío al este y 0,1 de quedarse en (0, 2) por el desvío al oeste, que choca con el borde.
- **Recompensas:** −0,04 por cada movimiento que termina en una celda no terminal; +1 al entrar a G = (3, 0) y −1 al entrar a X = (3, 1), en lugar de la recompensa por movimiento.
- **Terminales:** G y X. Al llegar ahí la misión termina y V(terminal) = 0.
- **Descuento:** γ = 0,9.

La posición basta porque las probabilidades del siguiente estado solo dependen de dónde está el dron y de qué acción toma: el viento actúa igual en cada paso y los muros y terminales son fijos. No importa por dónde llegó el dron a esa celda. Esto es la propiedad de Markov del modelo.

b) y c) Implementación

action_value recorre los pares (sucesor, probabilidad) de mdp.transitions y suma p × (R(s, a, s') + γ V(s')). Como el modelo ya entrega la recompensa terminal al entrar y V(terminal) = 0, esa recompensa se cuenta una sola vez.

value_iteration arma en cada barrido un diccionario nuevo: para cada estado no terminal toma el máximo de action_value sobre sus acciones, usando solo el diccionario del barrido anterior, y deja los terminales en cero. Luego calcula delta como el mayor cambio absoluto entre los dos diccionarios y marca converged si delta es menor que la tolerancia.

d) Control y propagación en cuadricula

En control (S y G en una fila, sin ruido) el primer barrido da V(S) = 1: la acción E entra directo a G y recibe +1, mientras que las otras chocan con el borde y valen −0,04 + 0,9 × 0 = −0,04. En el segundo barrido las otras acciones valen −0,04 + 0,9 × 1 = 0,86, pero E sigue en 1. El cambio máximo es 0 y converge en 2 barridos, como se esperaba.

En cuadricula estos son los primeros barridos (T es terminal y # es muro):

Barrido	y	x = 0	x = 1	x = 2	x = 3	Delta
1	0	−0,040	−0,040	0,792	T	0,792
	1	−0,040	#	−0,040	T	
	2	−0,040	−0,040	−0,040	−0,040	
2	0	−0,076	0,523	0,860	T	0,563
	1	−0,076	#	0,431	T	
	2	−0,076	−0,076	−0,076	−0,076	
3	0	0,323	0,673	0,908	T	0,399
	1	−0,108	#	0,522	T	
	2	−0,108	−0,108	0,256	−0,108	
4	0	0,464	0,735	0,921	T	0,281
	1	0,173	#	0,565	T	
	2	−0,138	0,125	0,316	0,039	

Converge en 26 barridos con tolerancia 10^{-8} y V(S) = 0,374. Los valores finales y la política se muestran en el punto 5.

Punto 5. Extracción y análisis de políticas

a) Implementación

extract_policy asigna None a los terminales. Para los demás estados evalúa action_value en el orden N, E, S, W y solo cambia la acción elegida si aparece un valor mayor por más de 10^{-12}. Ese

margen evita que una diferencia de redondeo rompa un empate real, de modo que en los empates queda la primera acción del orden. Esto pasa, por ejemplo, en el inicio de zona_riesgo, donde Q(N) y Q(S) valen ambos 1,003639 y la política escoge N.

Política en cuadricula (valor y acción):

y\x	0	1	2	3
0	0,610 E	0,766 E	0,928 E	G
1	0,487 N	#	0,585 N	X
2	0,374 N	0,327 E	0,428 N	0,189 W

Desde (3, 2), justo debajo de X, la política es W: se aleja de la pérdida aunque eso alargue el camino a G.

b) atajo con ruido 0 y 0,3

Estado	Ruido 0	Ruido 0,3
Inicio (0, 2)	E (V = 1,367)	N (V = 0,711)
Fila y = 2, celdas x = 1 a 5	E	N
(0, 3), esquina junto a las X	N	W

Sin ruido la ruta corta por la fila y = 2, justo encima de las X, no tiene riesgo, y la política va directo al este. Con ruido 0,3, moverse al este en esa fila tiene probabilidad 0,15 de desviarse al sur y caer en una X con −5. Por eso la política sube primero a la fila y = 1 y avanza por ahí. En (0, 3) la acción W choca con el borde, así que el dron se queda quieto con probabilidad 0,85 y nunca se mueve hacia la X que está a su derecha. Se analiza en el punto 6a, literal e.

c) zona_riesgo: recompensa por movimiento y descuento

Configuración	V(inicio)	Acción en el inicio	Acción en (3, 1), zona R
Original (paso −0,04, γ = 0,95)	1,004	N	N (sale de R)
Paso −0,30	−0,716	E	E (cruza R)
γ = 0,50	−0,073	N	N

Con paso −0,04 la política rodea la franja R por arriba o por abajo, porque dos pasos extra cuestan menos que la penalización de −0,40. Con paso −0,30 cada movimiento sale caro, rodear ya no compensa, y desde (3, 1) la política cruza la zona R hacia la meta. Con γ = 0,50 la meta, que está a seis pasos, casi no pesa en el valor del inicio: los valores lejanos quedan cerca de cero o negativos y casi iguales entre sí.

Punto 6. Reflexión sobre el trabajo realizado

6a. Análisis de los algoritmos y resultados

a) Evidencia, probabilidad y decisión. La evidencia cambia la probabilidad de incendio (0,10 sin evidencia, 0,67 con alarma y 0,012 sin alarma en sensor_basico) y con ella la acción: inspeccionar, brigada y monitorear. La probabilidad dice qué tan creíble es el incendio. La utilidad esperada combina esa probabilidad con el costo de equivocarse. Por eso, sin evidencia, conviene inspeccionar aunque el incendio solo tenga 10 % de probabilidad: no hacer nada cuesta −100 si hay fuego, y ese riesgo pesa más que el costo fijo de inspeccionar. En bosque, la alarma con falla da una probabilidad menor que la alarma sola, porque la falla explica la alarma.

b) Enumeración frente a eliminación. Los dos métodos calculan la misma suma de productos de la conjunta factorizada. Eliminación solo cambia el orden de las operaciones: suma cada variable oculta en cuanto deja de hacer falta, en vez de construir todos los mundos. Por eso deben coincidir, y en bosque con alarma los dos dan 0,329517. El orden sí cambia el trabajo: el orden topológico mantuvo factores de máximo 8 filas y el inverso llegó a 32, porque eliminar primero Temperatura juntó cinco variables en un solo producto. En redes grandes esa diferencia puede ser enorme.

c) Intentos y probabilidad de la evidencia. Con más intentos el error tiende a bajar (a priori pasó de errores cercanos a 0,01 con 1000 a cerca de 0,002 con 10000), aunque en una ejecución puntual no está garantizado. Con rechazo lo que importa son las muestras aceptadas, que son más o menos intentos × P(e). La alarma tiene P(e) cercana a 0,18 y deja unas 1800 de cada 10000. El caso rara exige Tormenta=1, Falla=1 y Alarma=1 al tiempo, con P(e) cercana a 0,0013, y deja apenas unas 13. Con tan pocas muestras la estimación es muy inestable, y con 1000 intentos muchas veces no hay estimación. Que no se acepten muestras no significa que la evidencia sea imposible, solo que es rara.

d) Barridos y criterio de parada. Los barridos muestran que el valor se propaga desde los terminales. En el primero solo la celda junto a G ve el +1 (0,792); en cada barrido siguiente esa información avanza una celda más, y el inicio, que está a cinco pasos de G, sigue en negativo hasta el barrido 4 y pasa a 0,083 en el barrido 5. Los cambios máximos bajan de forma geométrica (0,792, 0,563, 0,399, 0,281…) por el descuento. Se paró cuando el mayor cambio absoluto entre dos barridos fue menor que 10^{-8}, lo que pasó en el barrido 26, con un límite de 1000 barridos como respaldo.

e) Ruido, recompensa y descuento. El ruido hace que el modelo tenga en cuenta los desvíos: en atajo, con ruido 0,3, la política deja la ruta corta junto a las X y sube una fila, y el valor del inicio baja de 1,367 a 0,711. Una recompensa por movimiento más negativa empuja a terminar pronto aunque haya que pagar la penalización, como en zona_riesgo con −0,30, donde la política cruza la franja R. Un descuento bajo vuelve al dron miope: la meta lejana casi no cuenta y los valores se aplanan. La política no es una ruta fija: indica qué hacer en cada celda. Si el viento desvía al dron a una celda que no estaba en la ruta esperada, la política ya tiene la acción para ese caso.

6b. Reflexión sobre el uso de inteligencia artificial

Usamos Claude (Anthropic) para tres cosas: mejorar el código, proponer la estructura del informe y apoyar su redacción.

- **Mejora del código.** Primero desarrollamos nuestra propia versión de las funciones de los puntos 1 a 5 y después le pedimos a la herramienta que la revisara y la mejorara, sin cambiar firmas ni formatos de retorno. En el código entregado, nuestra versión inicial se conserva comentada y la versión activa, sin comentar, es la mejorada con ayuda de la IA.
- **Estructura del informe.** Le pedimos una propuesta de organización por puntos, con las tablas de resultados y el anexo de uso de IA.
- **Redacción del informe.** A partir de nuestros cálculos y de las salidas del programa, la herramienta nos ayudó a redactar los textos, que luego revisamos y ajustamos.

Comparar nuestra versión con la mejorada nos dejó varias lecciones. En enumeration_ask modificábamos evidence directamente y, si la consulta ya estaba observada, la sobrescribíamos; entendimos por qué hay que trabajar sobre copias del evento. En prior_sample usábamos random.random() en lugar del rng recibido, lo que impedía reproducir los resultados con la semilla. En sampling_ask dividíamos los conteos entre todos los intentos y no entre las muestras aceptadas. En value_iteration actualizábamos los valores sobre el mismo diccionario, lo que vuelve los barridos asincrónicos. Y en extract_policy elegíamos la acción por el valor del vecino, sin tener en cuenta el ruido ni las recompensas, cuando lo correcto es usar action_value. Aceptamos todas las mejoras propuestas porque cada una corregía un error o un caso que faltaba según el enunciado; no rechazamos ninguna.

Para verificar la solución comprobamos a mano el punto 1 (tabla conjunta, Bayes y utilidades) y el punto 3a (factores de cadena_humo) y los comparamos con la salida del programa. Revisamos que enumeración y eliminación dieran lo mismo en bosque, que la independencia de cadena_humo se viera en los números y que control diera V(S) = 1 en dos barridos. También probamos casos borde: evidencia imposible, consulta observada, pesos inválidos en normalize y cero muestras aceptadas. El detalle por función está en el anexo. Podemos explicar cada función en una sustentación.

Anexo. Uso de IA

Herramienta: Claude (Anthropic), usado como asistente en una conversación.

Usos y prompts principales, en resumen:

- **Código:** para cada función le pasamos nuestra versión junto con su docstring y le pedimos: “Revisa esta implementación, dinos qué errores o casos faltantes tiene según los tips del enunciado y mejórala sin cambiar la firma ni el formato de retorno”.
- **Estructura:** “Propón una estructura para el informe del taller que siga los puntos del enunciado, con tablas de resultados y el anexo de uso de IA”.
- **Redacción:** “Redacta el análisis de este punto a partir de nuestros cálculos y de estas salidas del programa”. Después revisamos y ajustamos cada texto.

A continuación, para cada función trabajada con IA, se presentan nuestra versión inicial, las sugerencias relevantes, los cambios aceptados, rechazados o adaptados y la forma de verificación. En el código entregado la versión inicial queda comentada justo encima de la versión final, que es la única activa.

normalize (algorithms/probability.py)

Versión inicial del equipo:

```
total = sum(weights.values())
return {outcome: weight / total for outcome, weight in weights.items()}
```

Sugerencias de la IA: nuestra versión no validaba la entrada: con un diccionario vacío retornaba {} sin avisar, con pesos que suman cero fallaba con ZeroDivisionError y aceptaba pesos negativos, infinitos o NaN. Sugirió rechazar el diccionario vacío, los pesos no numéricos (incluidos los booleanos), los negativos, los no finitos y la suma cero con ValueError, y usar math.fsum para reducir el error de redondeo.

Cambios: aceptados todos. Se conserva la idea original de dividir cada peso entre el total con un diccionario nuevo, lo que ya mantenía las claves y su orden.

Verificación: pruebas con {}, {"a": −1}, NaN, inf, {0, 0} y un peso de texto: todas lanzan ValueError. Con {"a": 1, "b": 3} retorna {0,25; 0,75} y el diccionario de entrada no cambia.

expected_utility (algorithms/probability.py)

Versión inicial del equipo:

```
total = 0
for s in distribution:
    total += distribution[s] * utilities[s]
return total
```

Sugerencias de la IA: la suma ponderada era correcta; sugirió reemplazar el ciclo por math.fsum sobre la misma suma para reducir el error de redondeo.

Cambios: aceptado. El cálculo es el mismo.

Verificación: las utilidades esperadas de las tres tablas del punto 1d coinciden con el cálculo manual (por ejemplo, EU(brigada) = −13,333 con alarma).

best_action (algorithms/probability.py)

Versión inicial del equipo:

```
values = {a: expected_utility(distribution, u) for a, u in actions.items()}
return max(values, key=values.get), values
```

Sugerencias de la IA: max conserva la primera acción en un empate exacto, pero una diferencia mínima de redondeo podía romper un empate real. Sugirió recorrer las acciones en orden y cambiar la elegida solo cuando haya una mejora estricta mayor que 10^{-12}.

Cambios: aceptado. El retorno (acción, diccionario de utilidades) no cambia.

Verificación: en los tres casos del punto 1d la acción elegida coincide con la de mayor utilidad; con dos acciones de igual utilidad retorna la primera de la tabla.

enumeration_ask (algorithms/inference.py)

Versión inicial del equipo:

```
def enumerate_all(variables, event):
    if not variables:
        return 1.0
    first, rest = variables[0], variables[1:]
    if first in event:
        return network.probability(first, event[first], event) * enumerate_all(rest,
event)
    total = 0.0
    for value in (0, 1):
        event[first] = value
        total += network.probability(first, value, event) * enumerate_all(rest,
event)
    del event[first]
    return total

weights = {}
for value in (0, 1):
    evidence[query] = value
    weights[value] = enumerate_all(network.variables, evidence)
del evidence[query]
return normalize(weights)
```

Sugerencias de la IA: nuestra versión modificaba evidence (le agregaba y luego le borraba la consulta) y modificaba el evento dentro de la recursión. Si la consulta ya estaba observada, la sobrescribía y al final la borraba de evidence. Además, con evidencia imposible fallaba con una división entre cero en vez de lanzar ValueError. Sugirió recorrer las variables por índice en orden topológico, extender siempre una copia del evento, dar peso cero al valor contrario cuando la consulta está observada y lanzar ValueError si los dos pesos suman cero.

Cambios: aceptados todos. Se conserva nuestra estructura recursiva (multiplicar si la variable está fijada y sumar sobre 0 y 1 si es oculta).

Verificación: coincide con el cálculo manual de Bayes en sensor_basico (0,666667) y con eliminación de variables en bosque con alarma (0,329517). Con la consulta observada retorna {0: 0, 1: 1}, evidence queda igual después de la llamada y una red de prueba con P(Alarma=1) = 0 lanza ValueError.

prior_sample (algorithms/sampling.py)

Versión inicial del equipo:

```
event = {}
for variable in network.variables:
    if random.random() < network.probability(variable, 1, event):
        event[variable] = 1
    else:
        event[variable] = 0
return event
```

Sugerencias de la IA: usábamos random.random() en vez del generador rng recibido, así que la semilla no controlaba las muestras. Sugirió usar solo rng.random() y escribir la asignación en una línea.

Cambios: aceptado. El recorrido en orden topológico y la consulta de la CPT con el evento parcial se mantienen.

Verificación: con la misma semilla se obtienen siempre las mismas muestras y estimaciones (por ejemplo, 0,0560 con semilla 7 y 1000 intentos a priori).

sampling_ask (algorithms/sampling.py)

Versión inicial del equipo:

```
counts = {0: 0, 1: 0}
for _ in range(samples):
    sample = prior_sample(network, rng)
    if all(sample[x] == v for x, v in evidence.items()):
        counts[sample[query]] += 1
return {v: c / samples for v, c in counts.items()}, sum(counts.values())
```

Sugerencias de la IA: nuestra versión dividía los conteos entre el total de intentos y no entre las muestras aceptadas, así que con rechazo la distribución no sumaba uno, y retornaba ceros cuando no se aceptaba ninguna muestra. Sugirió filtrar solo cuando reject es verdadero, contar las aceptadas aparte, normalizar los conteos con normalize y retornar (None, 0) si no hay aceptadas.

Cambios: aceptados todos.

Verificación: las estimaciones suman uno y se acercan al valor exacto al aumentar los intentos (tabla del punto 3e). En el caso rara con 1000 intentos y semilla 7 se aceptan 0 muestras y la función retorna (None, 0).

action_value (algorithms/planning.py)

Versión inicial del equipo:

```
total = 0
for successor, p in mdp.transitions(state, action):
    total += p * (mdp.reward(state, action, successor) + mdp.discount *
values[successor])
return total
```

Sugerencias de la IA: nuestra suma esperada era correcta; sugirió escribirla como una sola expresión con sum.

Cambios: aceptado. El cálculo es el mismo y no suma dos veces la recompensa terminal, porque V(terminal) = 0.

Verificación: en control, la acción E desde S vale 1 y las demás −0,04 en el primer barrido, como en el cálculo manual.

value_iteration (algorithms/planning.py)

Versión inicial del equipo:

```
delta = 0
for state in mdp.states:
    if mdp.actions(state):
        old = values[state]
        values[state] = max(action_value(mdp, state, a, values) for a in
mdp.actions(state))
        delta = max(delta, abs(values[state] - old))
converged = delta < tolerance
```

Sugerencias de la IA: nuestra versión actualizaba values sobre el mismo diccionario, así que algunos estados usaban valores del barrido actual (barridos asincrónicos), y solo calculaba delta

sobre los no terminales. Sugirió construir un diccionario nuevo en cada barrido usando solo el anterior, dejar los terminales en cero de forma explícita y calcular delta sobre todos los estados.

Cambios: aceptados todos. Se mantiene la condición de parada converged = delta < tolerance.

Verificación: control converge en 2 barridos con V(S) = 1. En cuadricula el primer barrido da −0,04 en todas las celdas salvo 0,792 junto a G, que es lo que corresponde a un barrido sincrónico; con nuestra versión inicial ese mismo barrido ya daba 0,434 en (2, 1) y 0,269 en (2, 2), porque usaba valores recién actualizados. Converge en 26 barridos con deltas decrecientes.

extract_policy (algorithms/planning.py)

Versión inicial del equipo:

```
policy = {}
for state in mdp.states:
    if not mdp.actions(state):
        policy[state] = None
        continue
    neighbors = {}
    for action in mdp.actions(state):
        dx, dy = {'N': (0, -1), 'E': (1, 0), 'S': (0, 1), 'W': (-1, 0)}[action]
        target = (state[0] + dx, state[1] + dy)
        neighbors[action] = values.get(target, values[state])
    policy[state] = max(neighbors, key=neighbors.get)
return policy
```

Sugerencias de la IA: nuestra versión elegía la acción que lleva al vecino de mayor valor, sin considerar el ruido del viento ni las recompensas. Sugirió evaluar cada acción con action_value en el orden N, E, S, W y cambiar la elegida solo con una mejora mayor que 10^{-12}, conservando None en los terminales.

Cambios: aceptados todos.

Verificación: comparamos ambas versiones sobre los mismos valores. En cuadricula, en (2, 0), junto a G, la versión por vecinos elegía N, porque V(G) = 0 y el “mejor vecino” era la propia celda al chocar con el borde; con action_value la política es E y entra a G. En atajo con ruido 0,3 la versión por vecinos seguía la ruta corta al este junto a las X, mientras que la final sube a la fila y = 1. En el inicio de zona_riesgo el empate entre N y S se resuelve con N.

No hubo sugerencias rechazadas ni adaptadas: aceptamos todas las anteriores. La única versión activa en el proyecto es la final.